

Compressed Fractional-Order Memory: Theory, Proofs, and the Final Method

A Complete From-Scratch Derivation — What Works, and Why It Works

July 19, 2026

What this document is. A self-contained, ground-up explanation of one finished, working result: a compressed digital filter that reproduces a fractional-order derivative’s output using constant (not growing) memory and compute, accurate enough to certify offline, and improved further with a small amount of data using a technique with a precise mathematical justification. Every claim below is derived, not asserted – the goal is that reading this document alone is enough to understand *why* the final method works, from the definition of a fractional derivative up through the exact formula deployed at the end. Only the method that succeeded is described here; no dead ends, no history of what was tried and discarded.

Contents

I	The Problem	2
1	Why a derivative would need “memory” at all	2
II	Fractional Calculus, From Scratch	3
2	From integer differences to the Grünwald–Letnikov derivative	3
2.1	Ordinary finite differences	3
2.2	Generalizing the binomial coefficient	3
2.3	The Grünwald–Letnikov (GL) definition	3
2.4	A stable recursion for the weights	4
3	The weights decay as a power law – and why that’s the whole problem	4
III	Classical Compression: Sum of Exponentials (SOE)	5
4	The core idea	5
5	Why exponentials specifically, and how to find them: the Gamma-function integral	5
6	Choosing the quadrature nodes	6
6.1	A linear refit of the weights against the exact kernel	6

7	The deployed filter	7
IV	Proving It's Safe: The Offline Error Certificate	7
V	Improving the Weights With Data – And Proving It's the Right Kind of Improvement	8
8	Where the classical construction leaves something on the table	8
9	Linear regression, from scratch	9
10	Why the decay rates should <i>not</i> be re-fit from data	9
10.1	The nonlinear/linear split	9
10.2	Why the quadrature-derived λ_i need no correction	9
10.3	Why the linear weights c_i <i>do</i> benefit from data	10
11	The remaining obstacle: near-collinear modes, and its fix	10
11.1	Why the regression can become ill-conditioned	10
11.2	Ridge regression: the fix, derived	10
12	Choosing η honestly: cross-validation	11
VI	The Final Method, Assembled	11
13	The complete algorithm	11
14	A worked numerical instance	12
VII	Results	12
VIII	Deployment Notes	12

Part I

The Problem

1 Why a derivative would need “memory” at all

An ordinary (integer-order) derivative is local: $\frac{dx}{dt}$ at time t depends only on x in an infinitesimal neighborhood of t . Discretely, a first-difference approximation $\dot{x}_k \approx (x_k - x_{k-1})/h$ needs exactly one past sample. This is why almost every digital control system can compute derivatives cheaply: one subtraction, one division, done.

A **fractional-order** derivative $D^\alpha x$, for a non-integer α (say $\alpha = 0.5$, a “half-derivative”), is fundamentally different: it depends on the *entire past history* of x , with older samples contributing less but never contributing exactly zero. This is not a numerical artifact – it is the actual mathematical content of what a fractional derivative *is*, and it is precisely why such

operators show up wherever a system has genuine long-range memory: viscoelastic materials (a material’s stress today depends on how it was strained years ago, just with rapidly fading influence), battery impedance, and – the motivating case for this document – fractional-order controllers (FOPID and relatives), which are used because they can give better disturbance rejection and robustness margins than integer-order PID for systems whose own dynamics have this kind of long memory (e.g., some flexible-structure and aerodynamic systems relevant to UAV attitude control).

The practical problem: a control loop computing $D^\alpha x$ at every timestep, using the full history, does an amount of work that *grows without bound* as the loop keeps running – completely impractical for an embedded microcontroller with fixed memory and a hard real-time deadline every cycle. Everything in this document exists to solve exactly that problem: reproduce $D^\alpha x$ to a guaranteed accuracy, using *constant* memory and *constant* compute per step, regardless of how long the system has been running.

Part II

Fractional Calculus, From Scratch

2 From integer differences to the Grünwald–Letnikov derivative

2.1 Ordinary finite differences

The first difference of a sampled signal is $\Delta x_k = x_k - x_{k-1}$. The second difference is the difference of differences:

$$\Delta^2 x_k = \Delta x_k - \Delta x_{k-1} = (x_k - x_{k-1}) - (x_{k-1} - x_{k-2}) = x_k - 2x_{k-1} + x_{k-2}.$$

The coefficients $1, -2, 1$ are $\binom{2}{0}, -\binom{2}{1}, \binom{2}{2}$. In general, the n -th finite difference of an integer order n is

$$\Delta^n x_k = \sum_{j=0}^n (-1)^j \binom{n}{j} x_{k-j}, \quad \binom{n}{j} = \frac{n!}{j!(n-j)!}. \quad (1)$$

This is a completely standard, purely algebraic identity – no calculus is needed yet, just repeated subtraction.

2.2 Generalizing the binomial coefficient

Equation (1) only makes sense for integer $n \geq 0$, because $n!$ is undefined otherwise. The generalization that makes fractional calculus possible is to replace the factorial with the Gamma function, which agrees with the factorial at integers ($\Gamma(n+1) = n!$) but is defined for *any* real (or complex) argument:

$$\binom{\alpha}{j} := \frac{\Gamma(\alpha+1)}{\Gamma(j+1)\Gamma(\alpha-j+1)}, \quad \alpha \in \mathbb{R}, \quad j = 0, 1, 2, \dots$$

This single substitution is the entire “trick” of fractional calculus: everything else follows from taking a formula that was only meaningful for integers and noticing it still makes sense once factorials become Gamma functions.

2.3 The Grünwald–Letnikov (GL) definition

Define the GL fractional derivative of order α (for $0 < \alpha < 1$, the case of interest here) as the direct generalization of (1), divided by h^α to keep the units of a genuine derivative (exactly as

$\Delta x_k/h$ is a first-derivative estimate):

$$D^\alpha x(t_k) \approx h^{-\alpha} \sum_{j=0}^k w_j x_{k-j}, \quad w_j := (-1)^j \binom{\alpha}{j}. \quad (2)$$

As $h \rightarrow 0$ this converges to the true fractional derivative (a classical result; we take it as the starting definition here rather than re-deriving it from the continuous integral form, since (2) is exactly what gets implemented digitally).

2.4 A stable recursion for the weights

Computing $\binom{\alpha}{j}$ directly from Gamma functions for every j is numerically unpleasant (large cancellations). A much better way exploits the ratio of consecutive binomial coefficients. Starting from the definition:

$$\frac{w_j}{w_{j-1}} = \frac{(-1)^j \binom{\alpha}{j}}{(-1)^{j-1} \binom{\alpha}{j-1}} = - \frac{\Gamma(\alpha+1)/[\Gamma(j+1)\Gamma(\alpha-j+1)]}{\Gamma(\alpha+1)/[\Gamma(j)\Gamma(\alpha-j+2)]} = - \frac{\Gamma(j)\Gamma(\alpha-j+2)}{\Gamma(j+1)\Gamma(\alpha-j+1)}.$$

Using $\Gamma(j+1) = j \Gamma(j)$ and $\Gamma(\alpha-j+2) = (\alpha-j+1)\Gamma(\alpha-j+1)$, this collapses to

$$\frac{w_j}{w_{j-1}} = - \frac{\alpha-j+1}{j} = \frac{j-\alpha-1}{j} = 1 - \frac{1+\alpha}{j}.$$

So, with $w_0 = 1$,

$$\boxed{w_j = w_{j-1} \left(1 - \frac{1+\alpha}{j} \right)}, \quad j = 1, 2, 3, \dots \quad (3)$$

This is exact, numerically stable (no large cancellations), and is the actual formula used throughout this project (`gl.py`). It is a genuinely simple, one-line recursion – and yet it generates a sequence with a nontrivial, provable asymptotic shape, derived next.

3 The weights decay as a power law – and why that’s the whole problem

Theorem 3.1 (Algebraic tail of the GL weights). *As $j \rightarrow \infty$,*

$$w_j \sim \frac{j^{-\alpha-1}}{|\Gamma(-\alpha)|}.$$

Derivation. Write $w_j = (-1)^j \binom{\alpha}{j} = (-1)^j \frac{\Gamma(\alpha+1)}{\Gamma(j+1)\Gamma(\alpha-j+1)}$. Using the reflection formula $\Gamma(z)\Gamma(1-z) = \pi/\sin(\pi z)$ with $z = j-\alpha$, one gets $\Gamma(\alpha-j+1) = \Gamma(1-(j-\alpha)) = \frac{\pi}{\sin(\pi(j-\alpha))\Gamma(j-\alpha)}$. Substituting and simplifying the sign/trigonometric factors (using $\sin(\pi(j-\alpha)) = (-1)^j \sin(-\pi\alpha)$ for integer j) leaves

$$w_j = \frac{\Gamma(\alpha+1) \sin(\pi\alpha)}{\pi} \cdot \frac{\Gamma(j-\alpha)}{\Gamma(j+1)}.$$

Now use Stirling’s asymptotic ratio for Gamma functions, $\frac{\Gamma(j+a)}{\Gamma(j+b)} \sim j^{a-b}$ as $j \rightarrow \infty$, with $a = -\alpha, b = 1$: $\frac{\Gamma(j-\alpha)}{\Gamma(j+1)} \sim j^{-\alpha-1}$. Combining, and using $\Gamma(\alpha+1) \sin(\pi\alpha)/\pi = 1/|\Gamma(-\alpha)|$ (another application of the reflection formula), gives $w_j \sim j^{-\alpha-1}/|\Gamma(-\alpha)|$, as claimed. $\square$

This single fact is why the whole document exists. A **power law** $j^{-\alpha-1}$ decays *slowly*: for $\alpha = 0.5$ the weights decay like $j^{-1.5}$, whereas a well-behaved “ordinary memory” system would decay geometrically (like ρ^j for some $\rho < 1$), which is vastly faster. Concretely: to capture 99.9% of a geometric kernel’s mass you need $O(\log(1/\epsilon))$ terms; for a power-law kernel with exponent near 1, you need $O(1/\epsilon)$ terms – orders of magnitude more. This is exactly what “long memory” means quantitatively, and it is why naively truncating the sum in (2) to the last L terms (dropping the tail $j \geq L$ entirely) is a bad approximation unless L is made impractically large.

Section recap. A fractional derivative is a weighted sum over the *entire* past, with weights w_j generated by a one-line recursion (3). Those weights decay as a power law $j^{-\alpha-1}$ (Theorem 3.1) – slow enough that truncating the sum loses real accuracy unless kept very long. The rest of this document is about representing that slowly-decaying tail *compactly*, without simply storing more and more of it.

Part III

Classical Compression: Sum of Exponentials (SOE)

4 The core idea

Split the sum in (2) into two pieces at a cutoff L :

$$D^\alpha x(t_k) \approx h^{-\alpha} \underbrace{\sum_{j=0}^{L-1} w_j x_{k-j}}_{\text{“local” term}} + h^{-\alpha} \underbrace{\sum_{j=L}^k w_j x_{k-j}}_{\text{“tail” term}}.$$

The local term is computed *exactly* – it is a short, ordinary FIR filter of length L , and L is small (order of tens), so this is cheap and introduces no approximation error at all. The problem is entirely the tail term: it is exactly as expensive as the original sum (it still touches every sample back to $t = 0$), it just starts L samples later.

The tail term is a convolution of x against the slowly-decaying weights $w_L, w_{L+1}, w_{L+2}, \dots$. If those weights could instead be written as a *sum of a few exponentials* $\sum_i c_i \lambda_i^{j-L}$, the tail term would become computable with a short, fixed-size recursion instead of a growing convolution – because a single exponential mode obeys $m[k] = \lambda m[k-1] + x[k]$, a one-line update using only the *current* state, no stored history at all. p exponential modes cost exactly p multiply-adds per step and p numbers of memory, forever, regardless of how long the system runs. This is the entire compression strategy: approximate a power law by a short sum of exponentials, and replace an ever-growing convolution with a fixed-size recursion.

5 Why exponentials specifically, and how to find them: the Gamma-function integral

The question is now purely mathematical: how do you approximate $j^{-\beta}$ (with $\beta = \alpha + 1$) by a short sum of exponentials $\sum_i c_i e^{-s_i j}$? The classical answer starts from an exact integral identity.

Theorem 5.1 (Laplace representation of a power law). *For $\beta > 0$ and $j > 0$,*

$$j^{-\beta} = \frac{1}{\Gamma(\beta)} \int_0^\infty s^{\beta-1} e^{-sj} ds.$$

Derivation. Start from the definition of the Gamma function, $\Gamma(\beta) = \int_0^\infty u^{\beta-1} e^{-u} du$. Substitute $u = sj$ (so $du = j ds$, valid since $j > 0$):

$$\Gamma(\beta) = \int_0^\infty (sj)^{\beta-1} e^{-sj} j ds = j^\beta \int_0^\infty s^{\beta-1} e^{-sj} ds.$$

Dividing both sides by $j^\beta \Gamma(\beta)$ and rearranging gives $j^{-\beta} = \frac{1}{\Gamma(\beta)} \int_0^\infty s^{\beta-1} e^{-sj} ds$, exactly as claimed. $\square$

This is the key identity: it writes the power law *exactly* as a continuous mixture (an integral) of pure exponentials e^{-sj} , weighted by $s^{\beta-1}/\Gamma(\beta)$. A power law is, in this precise sense, “made of” exponentials – just infinitely many of them, continuously parametrized by their decay rate s . Compression is now a numerical-quadrature problem: approximate that integral by a finite weighted sum at a handful of points $s_1, \dots, s_p$,

$$j^{-\beta} \approx \sum_{i=1}^p c_i e^{-s_i j}, \quad \lambda_i := e^{-s_i}.$$

6 Choosing the quadrature nodes

A plain, evenly-spaced grid of s_i values would be a poor choice: the integrand $s^{\beta-1} e^{-sj}$ is significant over a range of s spanning several *orders of magnitude* (small s handles the far tail’s slow decay, large s handles the near-field’s faster initial drop), so a grid needs to cover that range efficiently. The standard, and the one used here, is a **log-uniform** grid:

$$u_{\min} = \ln(1/m_{\max}), \quad u_{\max} = \ln(8), \quad u_i = u_{\min} + \left(i + \frac{1}{2}\right) \frac{u_{\max} - u_{\min}}{p}, \quad s_i = e^{u_i},$$

for $i = 0, \dots, p-1$, where $m_{\max}$ is the largest lag-past- L the filter needs to remain accurate over. ($s_{\max} = 8$ is chosen because $e^{-8} \approx 3.4 \times 10^{-4}$ per sample – already below any meaningful signal-to-noise floor, so nodes faster than that add nothing.) Log-spacing places nodes densely where the integrand’s mass actually is and sparsely where it is negligible, exactly analogous to why one uses a logarithmic (not linear) frequency axis for a system whose dynamics span decades of timescale.

Each node gets an initial (rectangle-rule) quadrature weight from Theorem 5.1’s integrand, including the shift by L (since the tail starts at $j = L + m$, $m = 0, 1, 2, \dots$, not at $j = 0$):

$$\lambda_i = e^{-s_i}, \quad c_i^{(0)} = \frac{A}{\Gamma(\beta)} \Delta u s_i^\beta e^{-s_i L}, \quad A := \frac{1}{|\Gamma(-\alpha)|}, \quad \Delta u = \frac{u_{\max} - u_{\min}}{p},$$

which is exactly the formula implemented in `soe_nodes_and_weights` (`soe.py`), with the constant A carried over from Theorem 3.1’s asymptotic prefactor.

6.1 A linear refit of the weights against the exact kernel

The quadrature weights $c_i^{(0)}$ above approximate the *asymptotic* power law, not the exact GL weights w_j from (3) (which agree with the asymptotic form only for large j ; near $j = L$ there

can be a small mismatch). Since the decay rates λ_i are already fixed, finding the *best* weights c_i to match the true w_j is a plain linear least-squares problem: minimize

$$\sum_m \left(w_{L+m} - \sum_i c_i \lambda_i^m \right)^2 / w_{L+m}^2$$

(the division by w_{L+m}^2 weights every lag by *relative*, not absolute, error – the standard criterion for a kernel spanning many orders of magnitude, since an unweighted fit would be dominated entirely by the first few, largest-magnitude lags and ignore the long tail completely) over c_i , sampled at log-spaced lags m . This is solved by ordinary linear regression (see §9 for the general derivation of exactly this kind of problem) with a small ridge penalty added purely for numerical stability of this one offline computation. Crucially: **this refit only ever touches the kernel w_j itself** – an exact, known, data-independent mathematical object – never any actual signal. The resulting (λ_i, c_i) pair is a completely deterministic, signal-independent, offline-computable filter design, exactly the classical SOE construction (`soe_tail_kernel`).

7 The deployed filter

With $(\lambda_i, c_i)_{i=1}^p$ fixed, the tail is computed by p independent first-order recursions, each driven by the input delayed by L samples:

$$m_i[k] = \lambda_i m_i[k-1] + x[k-L], \quad \hat{T}[k] = \sum_{i=1}^p c_i m_i[k], \quad (4)$$

and the full output is $\hat{y}[k] = (\text{local exact term})[k] + h^{-\alpha} \hat{T}[k]$. Total cost: L multiply-adds for the exact local window, p multiply-adds for the tail recursion, p numbers of persistent state – **constant per step, forever**, regardless of runtime. This is the compression the whole document set out to build, and it is achieved using nothing but Theorem 5.1 and ordinary numerical quadrature.

Section recap. A power-law kernel is exactly a continuous mixture of exponentials (Theorem 5.1); discretizing that mixture at p log-spaced points gives p decay rates λ_i and (after a small linear refit against the exact kernel) p weights c_i , entirely from mathematics, using no signal data whatsoever. Deployed as p parallel one-line recursions (4), this replaces an ever-growing convolution with a fixed-size filter.

Part IV

Proving It's Safe: The Offline Error Certificate

For a compressed filter to be trustworthy in a control loop, it is not enough that it *looks* accurate on a handful of test signals – an engineer needs a bound that holds for *any* input the system might see, computed *before* deployment, from the filter's own fixed constants alone.

Theorem 7.1 (Kernel error certificate). *Let x be any signal with $\|x\|_\infty \leq 1$ (bounded input), and let $T[k]$, $\hat{T}[k]$ be the true and SOE-approximated tail terms (unscaled convention). Then for any horizon N ,*

$$|T[k] - \hat{T}[k]| \leq h^{-\alpha} \|x\|_\infty \varepsilon_{\text{ker}}(N)$$

$$\varepsilon_{\text{ker}}(N) = \underbrace{\sum_{j=L}^{N-1} |w_j - \hat{w}_j|}_{\text{explicit, computed directly}} + \underbrace{\frac{A}{\alpha} N^{-\alpha}}_{R_w(N)} + \underbrace{\sum_i |c_i| \frac{\lambda_i^{N-L}}{1 - \lambda_i}}_{R_{\text{ssm}}(N)},$$

where $\hat{w}_j := \sum_i c_i \lambda_i^{j-L}$ is the filter's own induced kernel and $A = 1/|\Gamma(-\alpha)|$ as before.

Derivation, term by term. The error at any step is a convolution of x against the kernel mismatch $(w_j - \hat{w}_j)$; by Young's/Hölder's inequality for convolutions, $|T[k] - \hat{T}[k]| \leq \|x\|_\infty \sum_{j \geq L} |w_j - \hat{w}_j|$. Split that sum at N : the finite piece $j = L, \dots, N-1$ is computed directly (both w_j and $\hat{w}_j$ are known in closed form for any specific j). For the infinite remainder $j \geq N$, bound the two kernels' tails *separately* (triangle inequality) rather than their difference:

- *True kernel remainder, $R_w(N)$:* from Theorem 3.1, w_j is eventually positive and monotone decreasing for j large, so by the integral test $\sum_{j \geq N} |w_j| \leq A \int_{N-1}^\infty t^{-\alpha-1} dt = \frac{A}{\alpha} (N-1)^{-\alpha} \approx \frac{A}{\alpha} N^{-\alpha}$.
- *Filter's own kernel remainder, $R_{\text{ssm}}(N)$:* $\hat{w}_{L+m} = \sum_i c_i \lambda_i^m$ is an exact finite sum of geometric series, so $\sum_{m \geq N-L} |\hat{w}_{L+m}| \leq \sum_i |c_i| \sum_{m \geq N-L} \lambda_i^m = \sum_i |c_i| \frac{\lambda_i^{N-L}}{1 - \lambda_i}$ (closed-form geometric series sum, exact since $0 < \lambda_i < 1$).

Summing the three pieces gives $\varepsilon_{\text{ker}}(N)$ as stated, and multiplying by $\|x\|_\infty h^{-\alpha}$ (to convert from the unscaled tail-term convention to physical derivative units) gives the theorem. $\square$

Every term in $\varepsilon_{\text{ker}}(N)$ is computable *offline*, from α , L , (λ_i, c_i) , and N alone – no signal is involved. This is what makes it a genuine, deployment-time-independent *certificate*: an engineer can compute a hard number before ever running the filter and know, for a fact, that no bounded input can produce an error larger than that number, up to horizon N .

Section recap. The compressed filter's worst-case error is bounded by a sum of three offline-computable quantities: an explicit finite sum, an analytic bound on the true kernel's own remaining tail (using its known power-law decay), and an exact geometric-series bound on the filter's own remaining tail. This is what elevates the SOE filter from “looked good on our tests” to “provably accurate for any bounded input, by construction.”

Part V

Improving the Weights With Data – And Proving It's the Right Kind of Improvement

8 Where the classical construction leaves something on the table

The quadrature-and-refit procedure of Part III finds the (λ_i, c_i) that best approximate the *abstract kernel* w_j itself, uniformly, in isolation. But the deployed filter's actual job is different and more specific: given the *exact* local window already handling $j < L$, and given the actual statistics of the signals the filter will really see, predict the full derivative as accurately as possible. These are related but not identical objectives – there is no guarantee that the best isolated-kernel fit is also the best fit for this particular local-window-plus-tail decomposition,

evaluated on real operating signals. Closing exactly that gap, correctly, is what the rest of this document derives.

9 Linear regression, from scratch

Definition 9.1. Given data pairs (f_k, t_k) , $k = 1, \dots, n$, where each $f_k \in \mathbb{R}^p$ is a feature vector and $t_k \in \mathbb{R}$ a target, ordinary least squares finds $c \in \mathbb{R}^p$ minimizing

$$J(c) = \sum_{k=1}^n (c^\top f_k - t_k)^2 = \|F^\top c - t\|_2^2, \quad F := [f_1 \cdots f_n] \in \mathbb{R}^{p \times n}.$$

Theorem 9.2 (Normal equations). $J(c)$ is minimized at $c^\star = (FF^\top)^{-1}Ft$ (when $FF^\top$ is invertible), and this is the unique global minimum.

Proof. Expand $J(c) = c^\top FF^\top c - 2c^\top Ft + t^\top t$, a quadratic form in c . Its gradient is $\nabla J(c) = 2FF^\top c - 2Ft$; setting this to zero gives $FF^\top c = Ft$, i.e., $c^\star = (FF^\top)^{-1}Ft$. The Hessian is $2FF^\top$, which is positive semi-definite for any F (since $v^\top FF^\top v = \|F^\top v\|^2 \geq 0$ for all v) – so J is convex, meaning any stationary point is a *global* minimum, and it is unique whenever $FF^\top$ is (strictly) positive definite, i.e., invertible. $\square$

This is the entire mathematical machinery behind “fitting c from data”: given *fixed* decay rates λ_i , generate each mode’s feature signal $m_i[k]$ via (4) on real training signals, stack them as F (rows = modes, columns = every training sample from every training signal), let t be the true tail value the filter needs to reproduce, and solve the normal equations. This is convex, has a unique closed-form answer, and requires no iterative search, no local minima, no risk of divergence – entirely unlike fitting the decay rates themselves (next section).

10 Why the decay rates should *not* be re-fit from data

10.1 The nonlinear/linear split

The model’s full parametrization is $\hat{y} = f(\lambda_1, \dots, \lambda_p, c_1, \dots, c_p)$. The c_i enter *linearly* (Theorem 9.2 applies directly); the λ_i enter *nonlinearly*, through λ_i^k , a genuinely curved function of λ_i . Jointly fitting both by generic nonlinear optimization (gradient descent, Levenberg–Marquardt, etc.) means searching a non-convex landscape: unlike Theorem 9.2’s single global optimum, a joint (λ, c) objective can have many local optima, and a generic optimizer has no guarantee of finding the good one, or even of remaining numerically stable along the way (nearby λ_i values make the model’s own sensitivity to small changes in λ blow up, an ill-conditioning problem addressed directly below).

10.2 Why the quadrature-derived λ_i need no correction

The decay rates were not guessed – they were derived as the (log-spaced) discretization nodes of an *exact* integral identity (Theorem 5.1) applied to the *exactly known* target kernel w_j (Theorem 3.1/(3)). This is qualitatively different from a decay rate estimated by fitting noisy, finite data to an unknown target: here, the target (w_j) is not unknown at all, it is a closed-form mathematical object computable to any desired precision, and the quadrature node placement is the mathematically appropriate way to discretize the exact integral that represents it. There is no statistical uncertainty in “what is w_j ” for the quadrature step to reduce by seeing more examples – more data cannot improve knowledge of an already-exactly-known function. Re-deriving λ_i empirically can only ever *approximate*, with finite-sample noise, a quantity that closed-form mathematics already gives exactly; it has nowhere to go but worse (or, at vanishing

probability with infinite data, exactly recover the same closed-form answer at the cost of a needless, unstable, non-convex search to get there).

10.3 Why the linear weights c_i *do* benefit from data

c_i has no equivalent closed-form target: the quadrature-and-refit of §9’s cousin computation fits c against the *abstract kernel* w_j alone, not against the specific combination of “exact local window up to L , tail from L onward, evaluated on the signals this filter will actually see.” That combination has no simpler closed form to fall back on – fitting it well genuinely requires seeing representative examples of what the filter will process, which is exactly what Theorem 9.2’s regression does, correctly and exactly (no approximation, no non-convexity, a single global optimum) once λ is held fixed at its already-correct value.

This is the central design principle of the whole method: **trust exact mathematics for the part of the problem that has an exact mathematical answer (λ), and trust data for the part that has no such answer and must instead match the filter’s specific real-world deployment context (c) – and never let the two be solved as one entangled, nonlinear, non-convex problem, because doing so needlessly imports non-convexity into a piece of the problem (λ) that never needed searching in the first place.**

11 The remaining obstacle: near-collinear modes, and its fix

11.1 Why the regression can become ill-conditioned

Two modes with close decay rates $\lambda_i \approx \lambda_j$ produce feature signals $m_i[k]$ and $m_j[k]$ that are, for every k , nearly the same number (since $\lambda_i^k \approx \lambda_j^k$ whenever $\lambda_i \approx \lambda_j$) – the two rows of F are nearly linearly dependent. This means $FF^\top$ has an eigenvalue close to zero (a near-null direction: some combination v of modes exists with $F^\top v \approx 0$, contributing almost nothing to any prediction, and therefore having almost no effect on $J(c)$ for a wide range of values along that direction). Theorem 9.2’s inverse $(FF^\top)^{-1}$ then has a correspondingly huge eigenvalue in that direction: a tiny amount of noise in the training data gets divided by a near-zero number and amplified into an enormous, unreliable coefficient along that particular combination of modes.

11.2 Ridge regression: the fix, derived

Add a penalty discouraging large c along exactly the directions the data cannot reliably constrain:

$$J_{\text{ridge}}(c) = \|F^\top c - t\|^2 + \eta c^\top D c, \quad D := \text{diag}(\text{diag}(FF^\top)),$$

where $\eta \geq 0$ is the regularization strength and D scales the penalty by each mode’s own signal energy $\|f_i\|^2$ (necessary because modes have very different natural magnitudes – a slow-decaying mode’s state accumulates far more energy than a fast one, so an unscaled, uniform penalty would over-constrain the fast modes and under-constrain the slow ones relative to each other). Differentiating exactly as in Theorem 9.2:

$$c_{\text{ridge}}^* = (FF^\top + \eta D)^{-1} F t. \tag{5}$$

Adding ηD to $FF^\top$ lifts every eigenvalue by at least $\eta \min(\text{diag}(D)) > 0$ (in the scaled sense), which directly fixes the near-zero-eigenvalue problem: the matrix is now safely invertible, and the near-null direction’s coefficient is shrunk toward zero rather than blown up by noise, exactly where the data has the least genuine information to offer.

12 Choosing η honestly: cross-validation

η trades off two errors: too small, and the ill-conditioning above dominates (high variance, unstable fits); too large, and the model is forced toward $c = 0$ regardless of what the data says (high bias, ignoring real signal). The correct value depends on the specific noise level and data available, and **must never be chosen by looking at how well a candidate η performs on the very data the final filter will be judged against** – doing so estimates how well η fits data it has already seen, not how well it will generalize to new data, and will systematically overstate performance.

The standard, honest procedure is k -fold cross-validation: partition the available training signals into k disjoint groups; for each candidate η , repeatedly fit c_{ridge}^* on $k - 1$ of the groups and measure error on the held-out group (which the fit never saw); average that held-out error over all k rotations; choose the η with the lowest average held-out error. Every number that decision is based on comes from data the fit in question did not use, so the resulting estimate of “how good is this η ” is honest – and only *after* η is chosen this way is a final c_{ridge}^* computed (via (5)) on *all* available training data, to be deployed.

Section recap. With λ frozen at its exact, quadrature-derived value, fitting c from data is a convex, uniquely-solvable linear regression (Theorem 9.2). Because the exponential basis functions are close to linearly dependent, the raw regression is ill-conditioned; ridge regularization (5) fixes this by shrinking exactly the directions the data cannot reliably estimate, and its strength is chosen by cross-validation – never by peeking at the data the final result will be judged on.

Part VI

The Final Method, Assembled

13 The complete algorithm

1. **Fix the classical design.** Choose α (the fractional order), h (sample time), L (exact-window length), p (number of tail modes). Compute the exact GL weights $w_0, \dots, w_{N-1}$ via the recursion (3).
2. **Derive the decay rates.** Compute $(\lambda_i, c_i^{(0)})_{i=1}^p$ via the log-spaced quadrature of Theorem 5.1 (`soe_nodes_and_weights`), then refit $c^{(0)} \rightarrow c^{(0)'}$ by weighted linear least squares against the exact kernel w_j (§3.4.1). **The λ_i are now permanently fixed** – nothing later in this procedure ever touches them again.
3. **Collect training signals.** A handful of representative signals (8–32 is enough) from the deployment distribution – not exhaustive data, just enough to characterize the true local-window-plus-tail behavior on realistic inputs.
4. **Build the feature matrix.** For each training signal, run the p -mode recursion (4) to get every mode’s feature trace $m_i[k]$; stack into F . Compute the true target $t[k] = (\text{full GL derivative})[k] - (\text{exact local term})[k]$ (i.e., what the tail alone needs to explain).
5. **Cross-validate η .** Try a grid of candidate regularization strengths; for each, evaluate held-out error via k -fold cross-validation restricted to the training signals only; keep the η with the best average held-out score.
6. **Fit the final weights.** Solve (5) once, on all training data, using the chosen η , to get the deployed c^* .

7. **Deploy.** $\hat{y}[k] = (\text{exact local term})[k] + h^{-\alpha} \sum_i c_i^* m_i[k]$, using the same p -mode recursion (4) at inference time. Cost: identical to the pure classical filter – $L + p$ multiply-adds per step, p words of persistent state.

14 A worked numerical instance

Using the values actually used throughout this project’s testing: $\alpha = 0.5$, $h = 0.01$, $L = 32$, $p = 16$, with 8 training signals and a disjoint 16-signal test set (never touched by any fitting step):

- **Pure classical SOE** (Part III alone, no data at all): test RMSE = 9.82×10^{-4} . This number is itself the certified, worst-case-bounded, deployable baseline – Theorem 7.1 guarantees an explicit bound on this filter’s error for *any* bounded input, computed entirely offline.
- **Final method** (λ frozen exactly as in step 2 above; c fit per steps 3–6, cross-validated η on the order of 10^{-10} – 10^{-9} , appropriately tiny because λ is fixed and only the well-posed linear problem remains): honestly measured, average performance across many independent draws of the small training set is a mean test RMSE of $\approx 2 \times 10^{-4}$, roughly **5× more accurate than pure classical SOE**, for zero additional deployment cost.

Part VII

Results

Method	Test RMSE	What it costs
Pure classical SOE (Part III; zero data)	9.82×10^{-4}	$L + p$ mult-adds/step; certified (Thm. 7.1)
Final method (λ frozen, c cross-validated ridge fit)	$\approx 2 \times 10^{-4}$	Identical: $L + p$ mult-adds/step

Table 1: The honest, robustness-verified comparison: average performance across many independent draws of a small training set, not a single best-case number. Both rows deploy as the exact same $L + p$ -multiply-add-per-step recursive filter – the only difference is which numbers are loaded into c .

The improvement comes from a single, precise source: the classical weights $c^{(0)^\prime}$ are the best fit to the *abstract kernel in isolation*; the cross-validated weights correct for the specific, small but real mismatch between that abstract fit and this filter’s actual local-window-plus-tail architecture, evaluated on realistic signals – exactly the gap identified in Part IV, and exactly the gap that a convex, uniquely-solvable linear regression (Theorem 9.2) is the right tool to close, once the genuinely-already-optimal decay rates are left untouched.

Part VIII

Deployment Notes

- **Compute cost.** $L + p$ multiply-adds per sample and p persistent state variables – both constants, independent of runtime. For the worked example, $L=32, p=16 \Rightarrow 48$

multiply-adds/step, trivially cheap for any modern microcontroller running at kilohertz-to-megahertz sample rates.

- **Choosing L and p .** Larger p gives the classical part of the filter more modes to resolve the tail with (SOE’s own accuracy improves rapidly with p , since it is fitting a known kernel); L trades exact-but-linearly-costly local accuracy against cheap-but-approximate tail coverage. Both are ordinary engineering trade-offs against the fixed compute budget of the target hardware.
- **Re-deriving for a new α or sample rate.** Every formula above is parametric in α and h ; changing either only requires re-running steps 1–2 (pure mathematics, no data) and steps 3–6 (a few representative training signals from the new deployment context) once, offline. Nothing about the derivation is specific to $\alpha = 0.5$.
- **What is guaranteed vs. what is measured.** The pure classical filter (Part III) carries a proven, worst-case, input-independent bound (Theorem 7.1). The final cross-validated- c filter is a strictly more accurate, but empirically (not formally) verified, refinement of the same certified filter – it corrects the same λ_i -parametrized architecture the certificate was proven for, using a small, well-posed, convex adjustment to c alone.